



Parte 2: Arquitectura Stateless y Autenticación con JWT

Una vez consolidados los fundamentos de persistencia, encriptación y control de accesos basados en base de datos en la primera sección, el siguiente gran reto en el desarrollo de APIs REST modernas es optimizar la escalabilidad y el rendimiento del servidor. En aplicaciones tradicionales, el backend retiene el estado del usuario en memoria a través de sesiones; sin embargo, en entornos distribuidos, aplicaciones móviles o arquitecturas desacopladas con frontends independientes (como React, Angular o JavaScript vanilla), este enfoque satura los recursos y limita el crecimiento del sistema.

El objetivo de esta segunda guía práctica es evolucionar la API desarrollada previamente hacia una arquitectura completamente **sin estado (stateless)**. Para lograrlo, reemplazaremos el mecanismo de autenticación tradicional por **JWT (JSON Web Token)**, el estándar de la industria (RFC 7519) que permite transferir de forma segura e independiente la identidad y los privilegios del usuario en cada petición HTTP sin necesidad de que el servidor guarde un registro en memoria.

A través de un desarrollo guiado paso a paso, en esta segunda parte asimilarás los siguientes conceptos avanzados:

- **Estructura y Anatomía de un Token:** Comprensión de los componentes de un JWT (*Header*, *Payload* y *Signature*), firmas criptográficas HMAC con SHA-256 y el manejo de tiempos de expiración para mitigar riesgos de seguridad.
- **Construcción de Utilidades y DTOs de Autenticación:** Diseño de una clase utilitaria (`JwtUtil`) para la generación y validación matemática de tokens en *runtime*, junto con el uso de DTOs específicos para aislar el flujo de credenciales (`LoginRequest` / `AuthResponse`) de la lógica de negocio.
- **Filtros Personalizados e Inyección de Contexto:** Implementación de un interceptor de peticiones (`JwtAuthorizationFilter`) heredado de `OncePerRequestFilter` para extraer el token de la cabecera `Authorization` (esquema *Bearer Token*), verificar su autenticidad de forma manual y establecer dinámicamente el contexto de seguridad dentro de Spring Security.

Al finalizar esta guía, habrás transformado tu backend en un servicio ágil, seguro y completamente desacoplado, capaz de validar identidades en milisegundos sin depender de cookies ni sesiones de servidor, dejándolo listo para entornos modernos de alta demanda.

¿Qué es JWT y para qué lo usamos en Spring Security?

JWT (JSON Web Token) es un estándar para transmitir información segura entre dos partes (cliente y servidor) en forma de token. Se usa para autenticación en APIs REST, ya que evita que el servidor tenga que mantener estado (sin sesión).

El Token se genera tras autenticarse y contiene información codificada (como el usuario y roles), este token se envía en cada petición para que el backend valide que la petición es legítima y así no dependemos de sesión ni cookies, es un sistema *stateless* (sin estado).

En si, se genera un **token compacto y seguro** que se usa para transmitir información entre partes (cliente ↔ servidor) **de forma confiable**.

¿Por qué implementar JWT?

Spring Security por defecto maneja **sesiones de servidor**: cuando un usuario inicia sesión, se crea una sesión en el backend que se guarda en memoria. Esto funciona bien para aplicaciones tradicionales con HTML (como Thymeleaf), pero no es ideal para:

- Aplicaciones **frontend separadas** (React, Vue, Angular, etc.).
- Escenarios de **escalabilidad** (porque las sesiones viven en el servidor).
- APIs REST donde queremos un backend **sin estado (stateless)**.

JWT soluciona esto:

- Cada vez que el usuario se loguea (El usuario envía sus credenciales usuario y contraseña), se le entrega un **token JWT** que contiene su información (como nombre, rol, etc.).
- El **cliente (frontend)** guarda ese token y lo envía en cada petición.
- El **backend valida el token** y, si es válido, permite el acceso a los endpoints protegidos.
- No se necesita mantener una sesión en memoria: **todo es stateless**.

¿Qué contiene un JWT?

Un JWT se compone de **3 partes** codificadas en Base64, separadas por puntos (`.`): `HEADER.PAYLOAD.SIGNATURE`

Ejemplo:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJzdWIiOiJlc3VhcmI..."  
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

1 HEADER (Encabezado)

Describe **cómo se firma** el token y su tipo.

```
{  
  "alg": "HS256",    // Algoritmo de firma: HMAC con SHA-256  
  "typ": "JWT"       // Tipo: JWT  
}
```

Codificado en Base64 se convierte en algo como:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

2 PAYLOAD (Datos)

Aquí va la **información (claims)** que quieres transmitir. Ejemplo típico:

```
{  
  "sub": "user123",  
  "name": "Cristian Henao",  
  "role": "ADMIN",  
  "exp": 1718000000 // Expiración en formato UNIX timestamp  
}
```

Los claims más comunes:

- `sub`: subject (identificador del usuario)
- `exp`: expiración
- `iat`: emit at (fecha de emisión)
- `role`, `email`, `permissions`, etc.: personalizados

Codificado en Base64:

```
eyJzdWIiOiJlc2VyMTIzIiwibmFtZSI6IkpXVCJ9
```

3 SIGNATURE (Firma)

Se usa para **verificar que el token no ha sido modificado**.

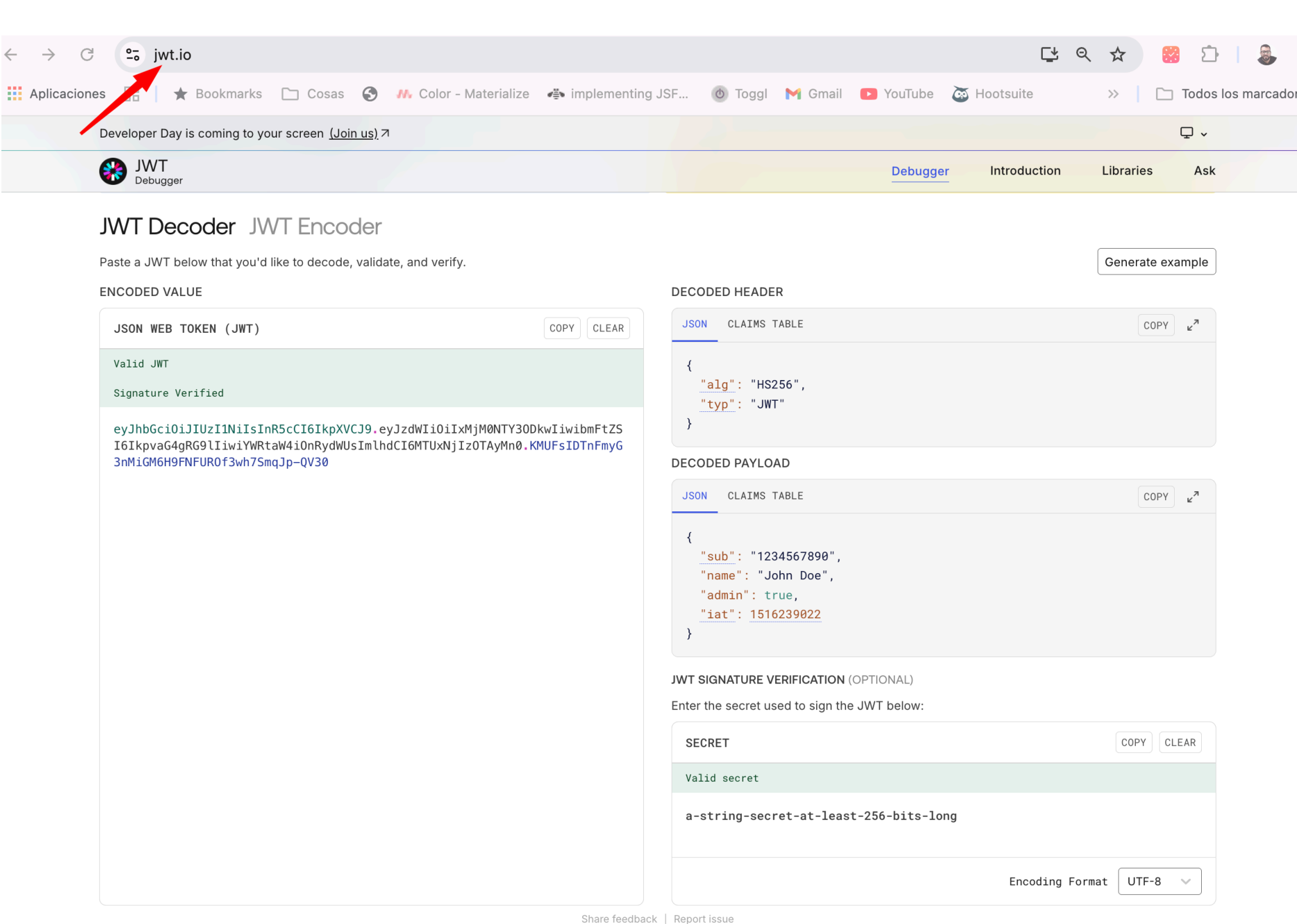
Fórmula:

```
firma = HMACSHA256(  
  Base64UrlEncode(header) + "." + Base64UrlEncode(payload),
```

```
clave_secreta
)
```

Así el servidor puede detectar si alguien manipuló el token.

Si se quiere profundizar más sobre cómo funcionan los tokens JWT, cómo se estructuran internamente o probar ejemplos en tiempo real, pueden visitar <https://jwt.io>. Allí encontrarán un visor interactivo que permite decodificar tokens, probar firmas y explorar bibliotecas JWT para distintos lenguajes de programación.



¿Qué necesitamos para usar JWT con JJWT?

Primero, tenemos que agregar la dependencia de JWT en el POM del proyecto para poder generar y validar los tokens JWT.

Agregar la dependencia de JWT en Maven

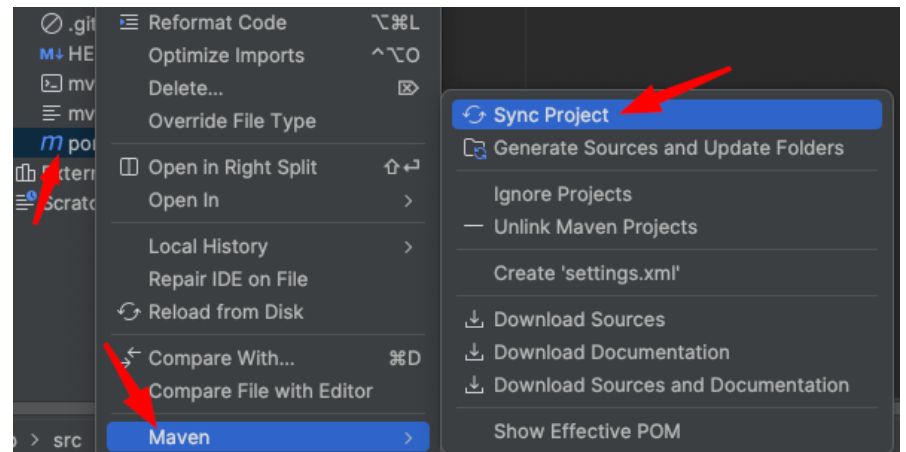
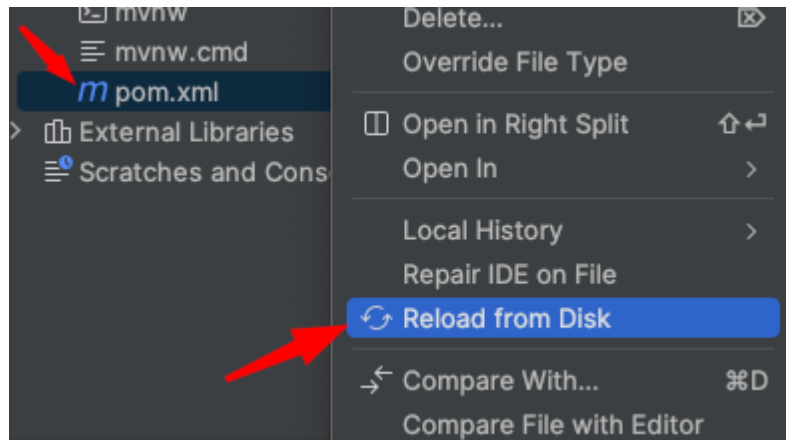
Abrimos el `pom.xml` y dentro de la sección `<dependencies>`, agregamos lo siguiente:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId> <!-- para serializar/deserializar JSON -->
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

Esto añade la librería JJWT y sus componentes necesarios para trabajar con JWT y JSON.

Nota: si no se reconocen las dependencias entonces se puede probar forzando al IDE a que actualice las dependencias sincronizando el proyecto o recargando.



Clase `JwtUtil`

Creamos la clase para generar y validar tokens JWT, Esta clase tendrá métodos para:

- **Generar un token** cuando el usuario se autentica.
- **Extraer información** (como el username) de un token.
- **Validar un token** para asegurarse que sea válido y no esté expirado.

Para esto creamos un paquete `jwt` en el paquete `security` y agregamos la clase con la siguiente información.

```
package com.veterinaria.jwtapp.security.jwt;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;

@Component
public class JwtUtil {

    // Llave secreta para firmar el token (debe ser larga y segura)
    private final Key claveSecreta = Keys.secretKeyFor(SignatureAlgorithm.HS256);

    // Tiempo de expiración del token (ejemplo: 1 hora)
    private final long expirationTime = 3600000; // 1 hora en ms

    // Método para generar token con el username como sujeto
    public String generarToken(String username) {
        return Jwts.builder()
            .setSubject(username) // poner el username como sujeto
            .setIssuedAt(new Date()) // fecha de creación
            .setExpiration(new Date(System.currentTimeMillis() + expirationTime)) // fecha
```

```

a de expiración
        .signWith(claveSecreta) // firmar con la clave secreta
        .compact();
    }

    // Obtener username del token
    public String obtenerUsername(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(claveSecreta)
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }

    // Validar token (ver si es válido y no expiró)
    public boolean validarToken(String token) {
        try {
            Jwts.parserBuilder()
                .setSigningKey(claveSecreta)
                .build()
                .parseClaimsJws(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            // Token inválido, expirado, etc
            return false;
        }
    }
}

```

Aquí podemos ver que se usa `private final Key claveSecreta = Keys.secretKeyFor (SignatureAlgorithm. HS256 );` con esto se está generando una clave secreta aleatoria para firmar los tokens con el algoritmo HMAC SHA-256. esta clave debe mantenerse **privada y segura**. (más adelante veremos como actualizar esta clave generada para que sea fija.)

adicional con `private final long expirationTime = 3600000;` definimos un tiempo limite de duración, en este caso de 1 hora.

Posteriormente se **genera el token** con la función `generarToken(String username)`

```

public String generarToken(String username) {
    return Jwts.builder()
        .setSubject(username) // El username va en el "sub" del token
        .setIssuedAt(new Date()) // Fecha de creación
        .setExpiration(new Date(System.currentTimeMillis() + expirationTime)) // Fecha de
        expiración (ej: 1 hora)
        .signWith(claveSecreta) // Firmar con la clave
        .compact(); // Construir el token como String
}

```

- Con esto se crea el JWT con los claims (información) mínima: sujeto, fecha y expiración.
- Se firma para evitar que alguien pueda modificarlo sin saber la clave.

Luego se **obtiene el username desde el token**

```

public String obtenerUsername(String token) {
    return Jwts.parserBuilder()

```

```

        .setSigningKey(claveSecreta)
        .build()
        .parseClaimsJws(token) // Parsear y validar firma
        .getBody()
        .getSubject(); // Sacar el campo "sub"
    }
}

```

Aquí se parsea el token, se verifica la firma y se obtiene el username guardado.

finalmente **se Valida token** si es valido o si ya se venció.

```

public boolean validarToken(String token) {
    try {
        Jwts.parserBuilder()
            .setSigningKey(claveSecreta)
            .build()
            .parseClaimsJws(token); // Lanza excepción si no es válido
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        // Token inválido o expirado
        return false;
    }
}

```

Se intenta validar el token, si algo falla (firma, expiración, formato), retorna `false`.

IMPORTANTE: si se reinicia la app, esta clave cambia porque se genera en runtime, por eso para producción se suele usar una clave fija (un secreto) cargada desde configuración.

Crear credenciales desde `application.properties`

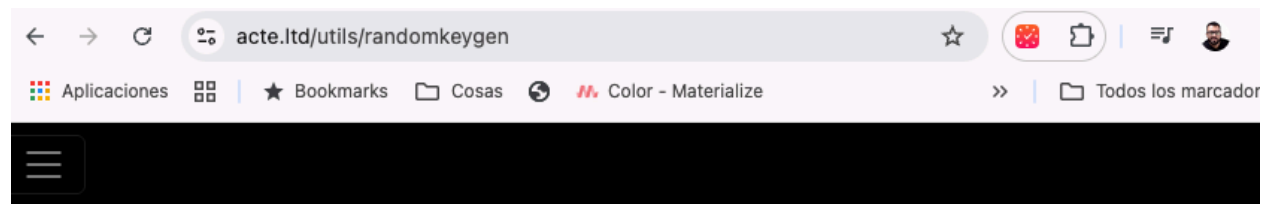
En el archivo properties agregamos lo siguiente

```

# JWT
jwt.clave.secret=CLAVE_EN_BASE64
jwt.expiration.time=3600000

```

y ahora para la clave secreta generaremos una de forma aleatoria, para eso podemos usar alguna página como <https://acte.ltd/utls/randomkeygen> donde copiamos alguna de las que nos arroje.



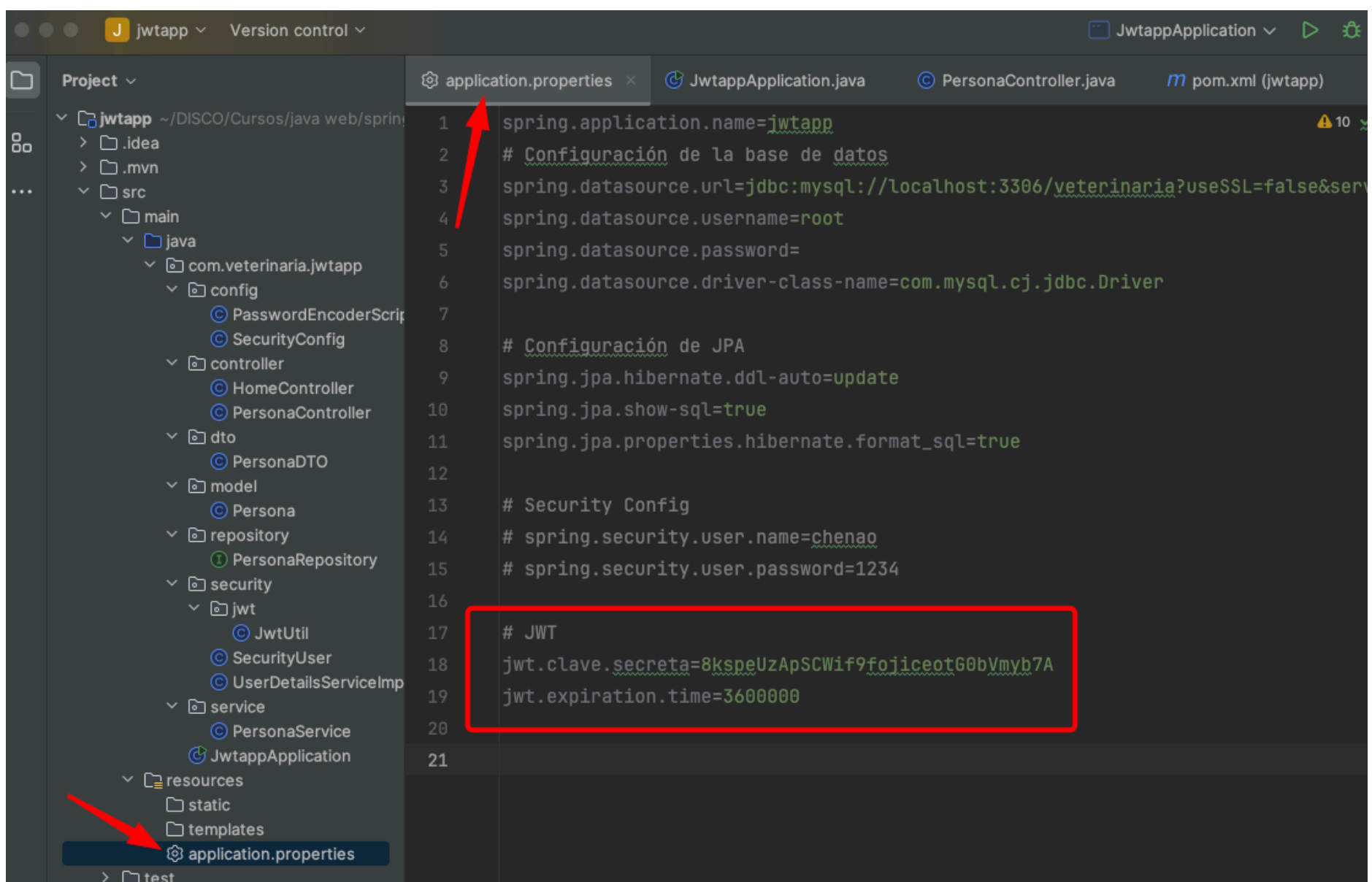
Random key generator

Password and secured key generator.

GENERATE

Basic 8	COPY	q4ITMSYe
Basic 10	COPY	FAnu5JCBmh
Basic 16	COPY	EZrmodOvegbH5MbO
Strong Not Ambiguous	COPY	9lEMfesHkSwS&FQQ
Strong	COPY	9qYelyk\$!S1%r'+
High	COPY	%".M+1{.;b{2:7+G-9#{tw9s>WLocjz
Encryption key 256	COPY	8kspeUzApSCWif9fojiceotG0bVmyb7A
64 wep	COPY	8l5DA
128 wep	COPY	2l8C5BEC68l74
256 wep	COPY	DlD747D483AEA6844l64E2CFBEA4B

al seleccionarla la copiamos a nuestro archivo quedando algo así



Posteriormente en `JwtUtil` actualizamos las variables de acceso a estas propiedades, de modo que ya se carguen desde el `application.properties`,

```
@Component 2 usages
public class JwtUtil {

    // Llave secreta para firmar el token (debe ser larga y segura)
    @Value("${jwt.clave.secreta}") 1 usage
    private String claveSecreta;

    // Tiempo de expiración del token (ejemplo: 1 hora)
    @Value("${jwt.expiration.time}") 1 usage
    private String expirationTime; // 1 hora en ms
```

asi mismo agregamos el método `obtenerClaveFirma()` para que se pueda encriptar la firma

```
//Obtener firma token
public Key obtenerClaveFirma(){
    return Keys.hmacShaKeyFor(claveSecreta.getBytes(StandardCharsets.UTF_8));
}
```

luego actualizamos los llamados en `generarToken(String username)`, `obtenerUsername(String token)` y `validarToken(String token)` para que llamen al metodo y no envíen la "claveSecreta" directamente, finalmente la clase queda así:

```
package com.veterinaria.jwtapp.security.jwt;

import io.jsonwebtoken.*;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.nio.charset.StandardCharsets;
import java.security.Key;
import java.util.Date;

@Component
public class JwtUtil {

    // Llave secreta para firmar el token (debe ser larga y segura)
    @Value("${jwt.clave.secreta}")
    private String claveSecreta;

    // Tiempo de expiración del token (ejemplo: 1 hora)
    @Value("${jwt.expiration.time}")
    private String expirationTime; // 1 hora en ms

    // Método para generar token con el username como sujeto
    public String generarToken(String username) {
        return Jwts.builder()
            .setSubject(username) // poner el username como sujeto
            .setIssuedAt(new Date()) // fecha de creación
            .setExpiration(new Date(System.currentTimeMillis()
                + Long.parseLong(expirationTime))) // fecha de expiración
            .signWith(obtenerClaveFirma(), SignatureAlgorithm.HS256) // firmar con la clav
```

```

    e secreta
        .compact();
    }

    //Obtener firma token
    public Key obtenerClaveFirma(){
        return Keys.hmacShaKeyFor(claveSecreta.getBytes(StandardCharsets.UTF_8));
    }

    // Obtener username del token
    public String obtenerUsername(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(obtenerClaveFirma())
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }

    // Validar token (ver si es válido y no expiró)
    public boolean validarToken(String token) {
        try {
            Jwts.parserBuilder()
                .setSigningKey(obtenerClaveFirma())
                .build()
                .parseClaimsJws(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            // Token inválido, expirado, etc
            return false;
        }
    }
}

```

Ahora vamos a mejorar el acceso a la aplicación usando la generación de Tokens.

Hasta este punto tenemos lo siguiente:

Ya contamos con la clase `JwtUtil` que se encarga de generar y validar tokens JWT usando la clave secreta. Esto es fundamental porque el token será nuestra forma de autenticar al usuario en las peticiones posteriores.

Tenemos la entidad `Persona` y un `PersonaDTO` que expone muchos datos, incluida la contraseña y el rol, en un solo objeto. Esto puede ser un riesgo porque no queremos exponer la contraseña ni enviar más datos de los necesarios en la respuesta del login.

Ahora necesitamos crear un **filtro** para interceptar las peticiones, validar el token y establecer el contexto de seguridad en Spring, pero para poder crear el Filtro vamos a introducir algunos cambios para mejorar la seguridad y claridad del flujo de autenticación JWT.

Paso 1: Crear los DTOs para autenticación

Para manejar correctamente el login con JWT, necesitamos definir claramente qué datos vamos a recibir y qué datos vamos a devolver en las peticiones relacionadas con autenticación.

¿Por qué usar DTOs?

- Para **no exponer directamente** la entidad `Persona` o el `PersonaDTO` actual (que puede tener campos sensibles como la contraseña).
- Para tener una estructura clara y limitada de datos que entran y salen por la API.

- Para que el controlador y servicio trabajen con datos específicos de autenticación, separados del resto.

1.1 Crear `LoginRequestDTO`

Este DTO contendrá los datos que el usuario enviará para iniciar sesión, típicamente:

```
package com.veterinaria.jwtapp.dto;

public class LoginRequestDTO {
    private String correo; // Usuario que usas para login
    private String password; // Contraseña en texto plano que ingresa el usuario

    // Getters y setters
    public String getCorreo() {
        return correo;
    }
    public void setCorreo(String correo) {
        this.correo = correo;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

1.2 Crear `LoginResponseDTO`

Este DTO será lo que respondemos al cliente después de un login exitoso. Por ejemplo, podemos devolver:

- El token JWT generado.
- El nombre o correo del usuario autenticado.
- El rol, para que el frontend conozca los permisos.

Ejemplo básico:

```
package com.veterinaria.jwtapp.dto;

public class LoginResponseDTO {
    private String token;
    private String correo;
    private String rol;

    //Si queremos devolver solo el token
    public LoginResponseDTO(String token) {
        this.token = token;
    }

    public LoginResponseDTO(String token, String correo, String rol) {
        this.token = token;
        this.correo = correo;
        this.rol = rol;
    }

    // Getters y setters
}
```

```

    public String getToken() {
        return token;
    }
    public void setToken(String token) {
        this.token = token;
    }
    public String getCorreo() {
        return correo;
    }
    public void setCorreo(String correo) {
        this.correo = correo;
    }
    public String getRol() {
        return rol;
    }
    public void setRol(String rol) {
        this.rol = rol;
    }
}

```

Paso 2: Agregar método para autenticar en **PersonaService**

El objetivo aquí es crear un método que reciba el correo y la contraseña que vienen en el `LoginRequestDTO`, valide que existan y que la contraseña coincida (usando `PasswordEncoder`), y si todo es correcto, devuelva la entidad `Persona` para luego generar el token JWT.

2.1 Método **autenticar** en **PersonaService**

Agrega este método en tu clase `PersonaService`:

```

public Optional<Persona> autenticar(String correo, String password) {
    Optional<Persona> personaOpt = personaRepository.findByCorreo(correo);

    if (personaOpt.isPresent()) {
        Persona persona = personaOpt.get();
        // Verificamos la contraseña usando el PasswordEncoder
        if (passwordEncoder.matches(password, persona.getPassword())) {
            return Optional.of(persona);
        }
    }
    return Optional.empty(); // Usuario no existe o contraseña incorrecta
}

```

¿Qué hace este método?

- Busca la persona por correo en la base de datos.
- Si existe, compara la contraseña que llega (en texto plano) con la contraseña guardada (encriptada).
- Si coincide, retorna la persona.
- Si no, retorna vacío para indicar fallo.

Importante

- Este método solo se encarga de validar usuario y contraseña.
- La generación del token JWT (con `JwtUtil`) y la creación del DTO de respuesta la hacemos después, en el controlador o en otro servicio.

Paso 3: Agregar el controlador para el login

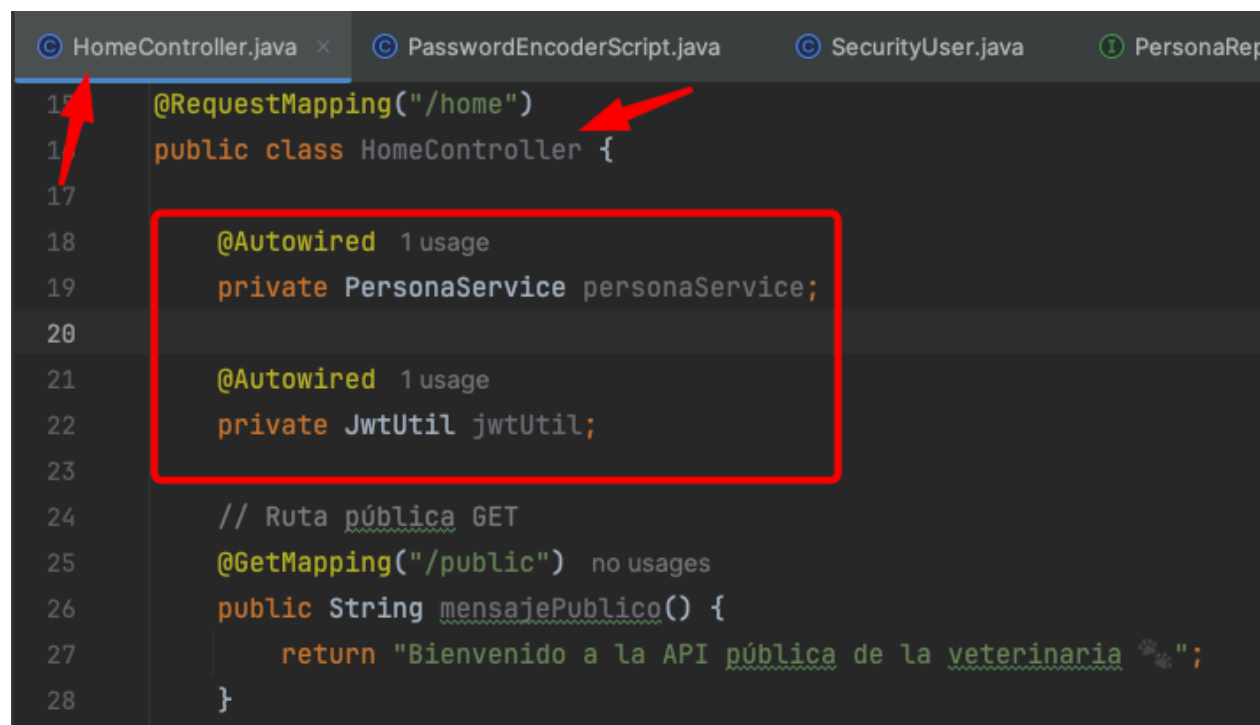
Este controlador tendrá un endpoint tipo POST donde el cliente envíe el `LoginRequestDTO` con correo y contraseña.

Si la autenticación es exitosa, devolvemos un `LoginResponseDTO` con el token generado.

3.1 Actualizamos la clase `HomeController`

Esta clase es la que usamos como controlador para las Api's publicas, que no requieren autorización, como en este caso el Login lo pueden hacer todos los usuarios entonces configuramos aquí el acceso y modificamos el método de prueba para el login que ya habíamos creado con anterioridad.

Inicialmente inyectamos `PersonaService` y `JwtUtil`



```
15 @RequestMapping("/home")
16 public class HomeController {
17
18     @Autowired 1 usage
19     private PersonaService personaService;
20
21     @Autowired 1 usage
22     private JwtUtil jwtUtil;
23
24     // Ruta pública GET
25     @GetMapping("/public") no usages
26     public String mensajePublico() {
27         return "Bienvenido a la API pública de la veterinaria 🐾";
28     }
```

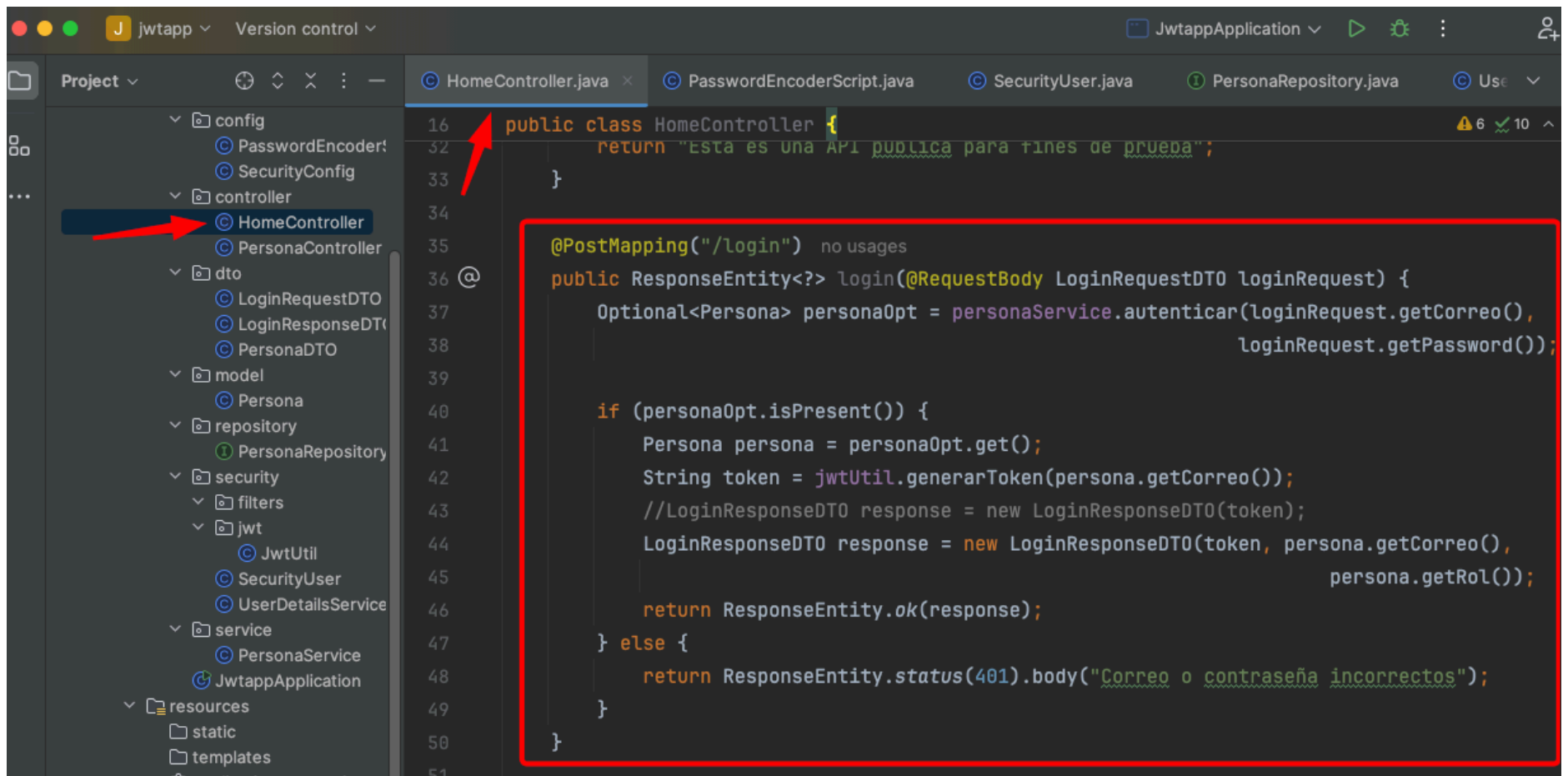
y modificamos el servicio de login

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequestDTO loginRequest) {
    Optional<Persona> personaOpt = personaService.autenticar(loginRequest.getCorreo(),
                                                             loginRequest.getPassword());

    if (personaOpt.isPresent()) {
        Persona persona = personaOpt.get();
        String token = jwtUtil.generarToken(persona.getCorreo());
        //LoginResponseDTO response = new LoginResponseDTO(token);
        LoginResponseDTO response = new LoginResponseDTO(token, persona.getCorreo(),
                                                         persona.getRol());

        return ResponseEntity.ok(response);
    } else {
        return ResponseEntity.status(401).body("Correo o contraseña incorrectos");
    }
}
```

quedando así



Explicación:

- El endpoint recibe un JSON con `correo` y `password`.
- Llama al método `autenticar` de `PersonaService`.
- Si la persona es válida, genera el token JWT con `JwtUtil`.
- Devuelve un `LoginResponseDTO` con el token.
- Si falla, responde con status 401 y un mensaje de error.

NOTA: Como se puede ver, se tiene comentado un response que solo crea el objeto con el token, este no enviaria información de correo ni rol, mientras que el otro que realiza el llamado al constructor con los 3 parámetros si los envía.

Prueba del servicio.

Ahora ya con esto podemos probar nuestro inicio de sesión y generación del token, para esto desde postman debemos llamar al endpoint

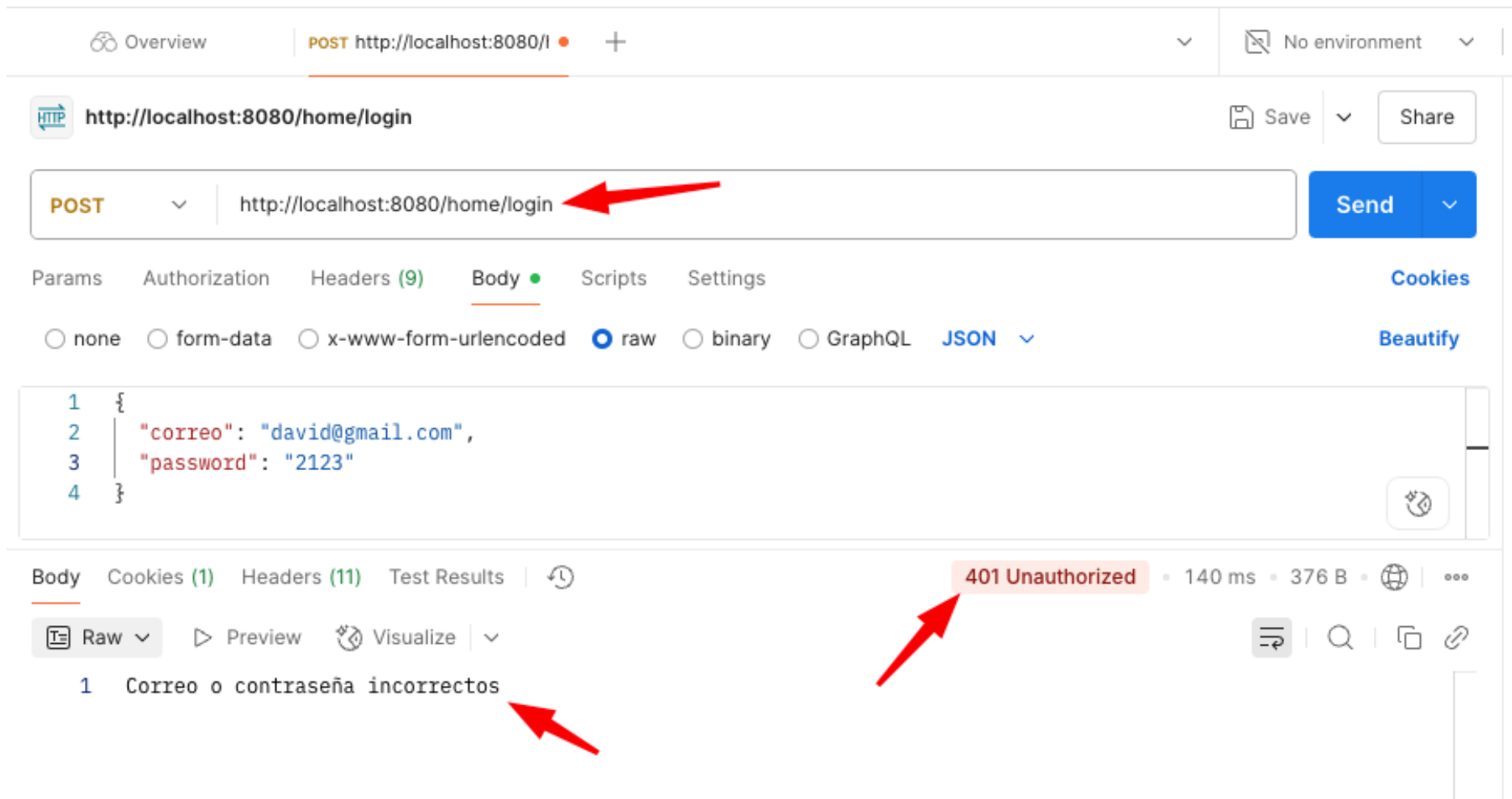
http://localhost:8080/auth/login y le enviamos en el body un json con esta estructura

```

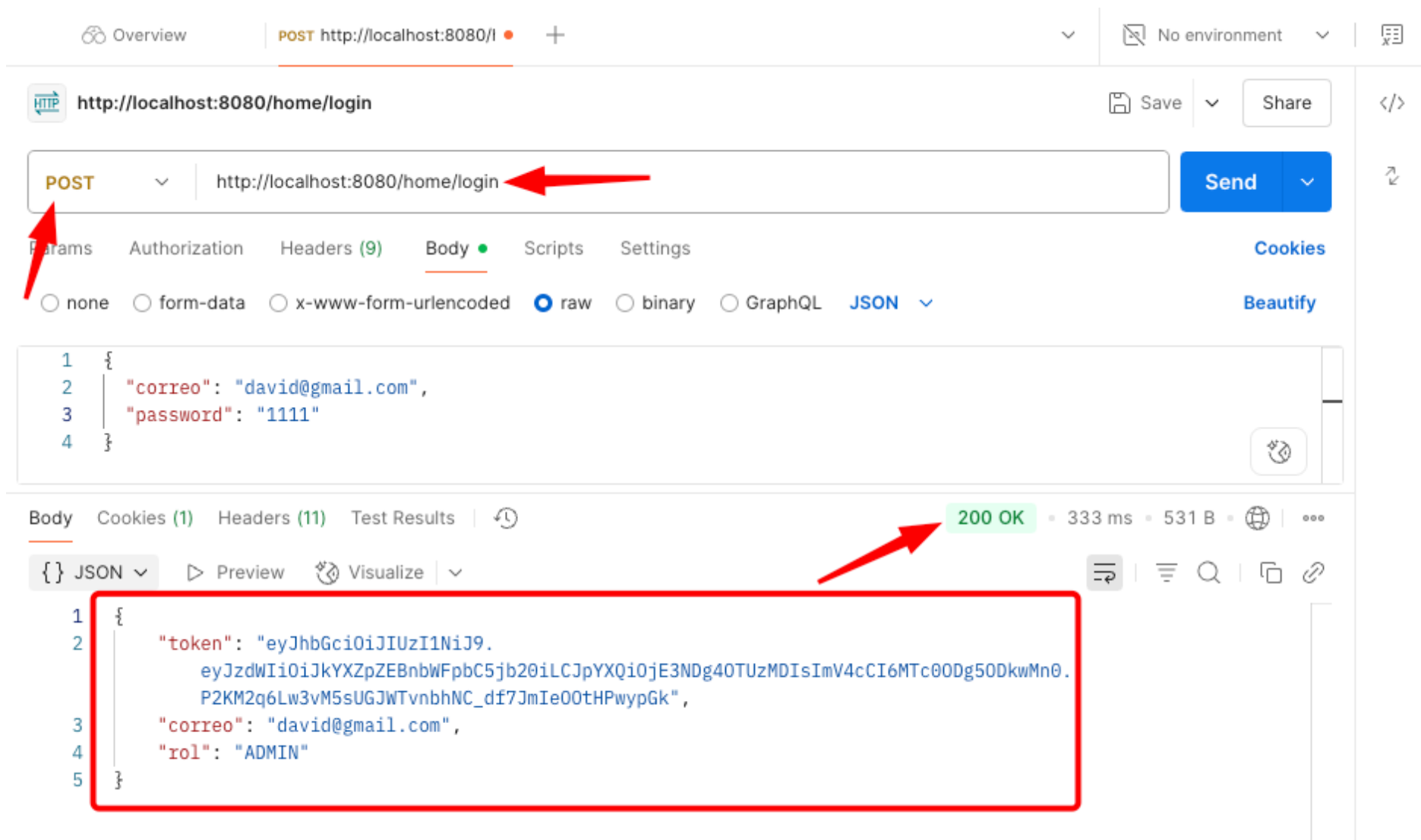
{
  "correo": "usuario@correo.com",
  "password": "la_contraseña"
}

```

de la siguiente manera, inicialmente con un usuario valido pero contraseña invalida



Vemos como nos devuelve un estatus 401 y el mensaje de correo o contraseña incorrectos.
Ahora enviamos la contraseña correcta



y aqui vemos como se devuelve el token y la información del usuario.

Ahora debemos crear el filtro que interceptará las solicitudes de inicio de sesión.

¿Qué es un filtro en Spring Security?

Un **filtro** es una clase que **intercepta las peticiones HTTP** antes de que lleguen a los controladores o después de salir de ellos.

En Spring Security, los filtros permiten ejecutar lógica como autenticación o autorización **de forma centralizada y automática** para cada solicitud.

En una aplicación que usa JWT con Spring Security, normalmente debemos implementar **dos filtros personalizados** que controlan la seguridad en distintos momentos del flujo:

- Uno para generar el token cuando el usuario inicia sesión.
- Otro para validar ese token en cada solicitud protegida.

Estos filtros son clave porque **Spring Security no gestiona JWT de forma nativa**, así que nosotros debemos encargarnos de interceptar y procesar el token manualmente, mediante 2 filtros principales.

- `JwtAuthenticationFilter` → Intercepta el login. Toma las credenciales del cuerpo de la solicitud (`username` y `password`), las valida y, si son correctas, **genera y devuelve un JWT** al cliente. (este sería opcional si vamos a controlar el login mediante API's)
- `JwtAuthorizationFilter` → Se ejecuta en cada petición que requiere autenticación. **Extrae y valida el JWT** desde el encabezado `Authorization`. Si el token es válido, permite el acceso y configura el usuario autenticado en el contexto de seguridad de Spring.

En resumen...

Si hacemos nuestro **propio login con un controlador** (`@PostMapping("/login")`), **NO necesitamos** `JwtAuthenticationFilter` , Pero **SIEMPRE necesitamos** `JwtAuthorizationFilter` para proteger las rutas privadas usando JWT.

¿Por qué necesitamos el filtro JWT si ya tenemos el login?

Hasta ahora ya hicimos que el usuario pueda **loguearse** enviando su correo y contraseña, y si son correctos, le entregamos un **token JWT**.

Pero, ¿qué pasa cuando ese usuario quiere usar otros endpoints que sí están protegidos? **El login** es para comprobar quién ingresa **una sola vez** y entregarle un token que diga "Este usuario es válido".

Después, para usar otros servicios, **no se debe enviar el usuario y contraseña otra vez**, porque es inseguro y poco práctico, en lugar de eso, se envía el **token** que se recibe en el login. Ese token es el "pase" o "credencial" para que el sistema sepa quién ingresa sin pedir la contraseña de nuevo.

¿Y cómo sabe el sistema quién es con solo el token?

Aquí entra en acción el **filtro JWT** (`JwtAuthorizationFilter`):

- Cada vez que se hace una petición a un endpoint protegido, el filtro revisa si en la petición viene el token JWT.
- Si encuentra el token, lo valida (verifica que no esté expirado y que no haya sido modificado).
- Si el token es válido, extrae el usuario y sus roles del token y se lo pasa a Spring Security para que permita o niegue el acceso según los permisos.
- Si el token no es válido o no viene, la petición es rechazada (por ejemplo, con un error 401 o 403).

Entonces...

- El **login** sirve para **generar el token** (solo se valida usuario y contraseña aquí).
- El **filtro JWT** sirve para **validar el token en todas las peticiones siguientes** y garantizar que solo los usuarios autenticados y autorizados accedan.

Creamos la clase `JwtAuthorizationFilter`

Ahora pasamos a la creación del `JwtAuthorizationFilter`, que se encarga de interceptar todas las solicitudes que intentan acceder a recursos protegidos. Su función principal es procesar el token JWT que llega en el encabezado de la solicitud, validarlo y, si es correcto, establecer el contexto de seguridad para que Spring Security reconozca al usuario como autenticado. Esta clase permite:

- Interceptar las solicitudes protegidas para extraer el token JWT del encabezado.

- Validar que el token JWT sea válido y no haya expirado.
- Obtener la información del usuario contenida en el token.
- Establecer el contexto de seguridad con los datos del usuario autenticado para permitir el acceso a los recursos.
- Bloquear o rechazar la solicitud si el token no es válido o está ausente.

Para esto creamos un paquete `filters` dentro del paquete `security` y agregamos la clase con la siguiente información y en el

```
package com.veterinaria.jwtapp.security.filters;

import com.veterinaria.jwtapp.security.UserDetailsServiceImpl;
import com.veterinaria.jwtapp.security.jwt.JwtUtil;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthorizationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private UserDetailsServiceImpl userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String authorizationHeader = request.getHeader("Authorization");

        if (authorizationHeader != null && authorizationHeader.startsWith("Bearer ")) {
            String token = authorizationHeader.substring(7);
            if (jwtUtil.validarToken(token)) {
                String correo = jwtUtil.obtenerUsername(token);
                UserDetails userDetails = userDetailsService.loadUserByUsername(correo);

                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(userDetails, null, userDetails
                        .getAuthorities());

                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

```

    }

    filterChain.doFilter(request, response);
}
}

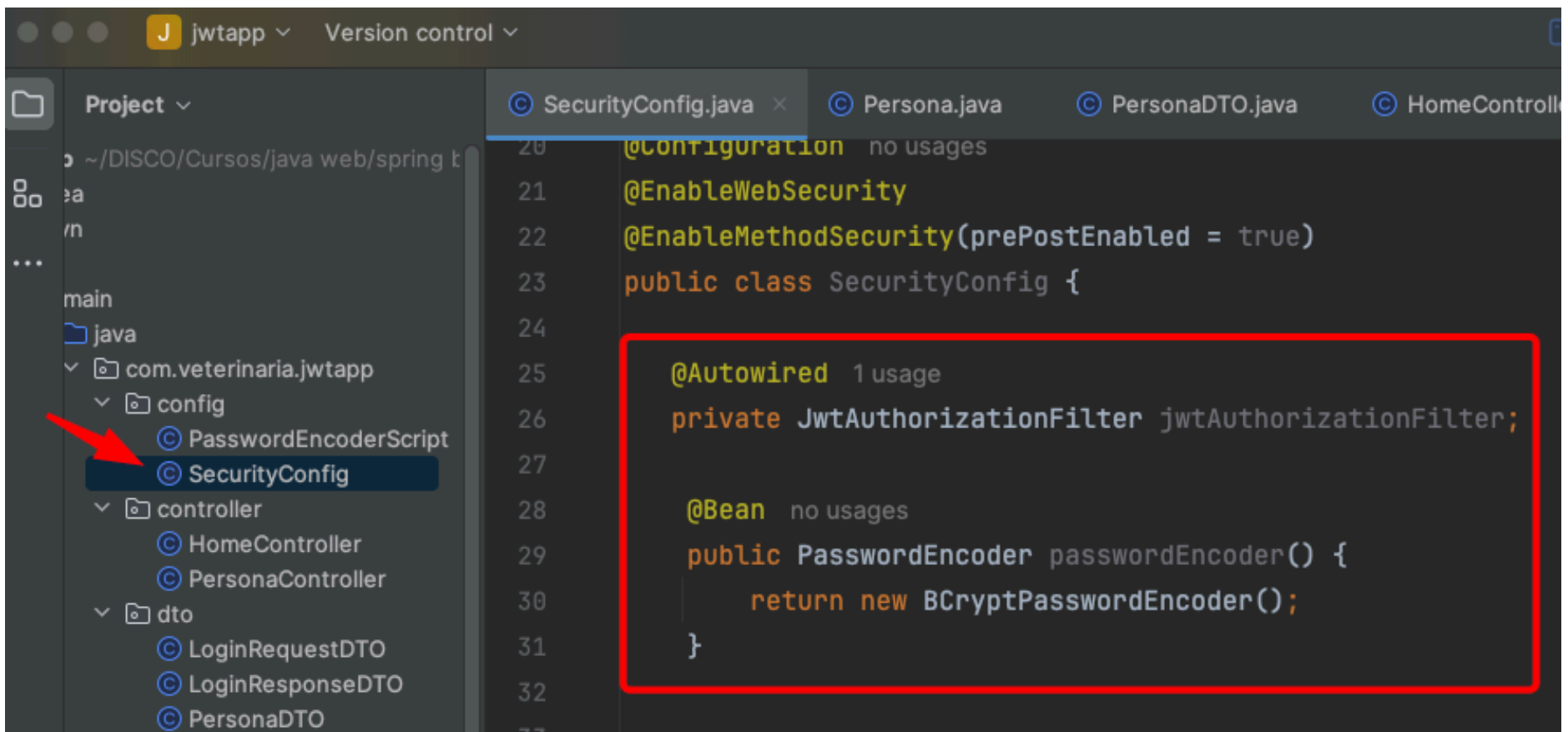
```

- Esta clase intercepta cada petición.
- Busca un token JWT en el encabezado Authorization.
- Si el token es válido, autentica al usuario dentro de Spring Security.
- Esto permite que el sistema controle el acceso a los endpoints protegidos según los roles del usuario.
- Si no hay token o es inválido, la petición sigue sin autenticación y puede ser bloqueada por Spring Security más adelante.

Actualizamos **SecurityConfig**

Ahora debemos actualizar la clase `SecurityConfig` agregando la referencia a `JwtAuthorizationFilter` y actualizando el método `securityFilterChain(HttpSecurity http)`

Primero inyectamos `JwtAuthorizationFilter` así:



y finalmente actualizamos el método `securityFilterChain` agregando

```

.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // sin
estado.addFilterBefore(jwtAuthorizationFilter, UsernamePasswordAuthenticationFilter.class) // añadir filtro JWT

```

quedando de la siguiente manera

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/personas/**").authenticated()
            .requestMatchers("/home/**").permitAll()
            .anyRequest().authenticated() // otras rutas deben ser autenticadas
        )
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // sin estado
}

```

```

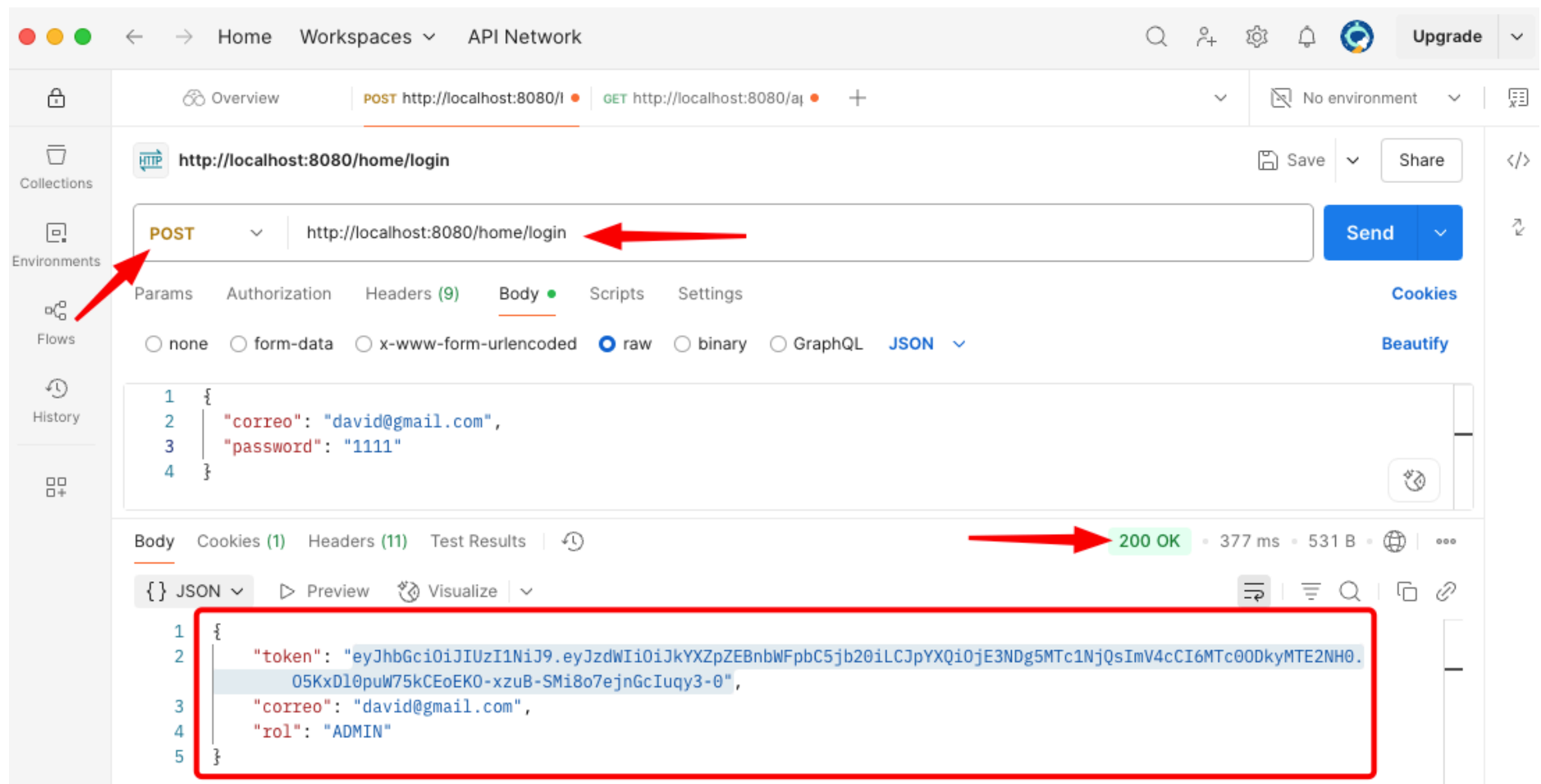
        .addFilterBefore(jwtAuthorizationFilter, UsernamePasswordAuthenticationFilter.class) // añadir filtro JWT
        //.httpBasic(Customizer.withDefaults()) // quitar httpBasic para no usar Basic Auth

        //.formLogin(Customizer.withDefaults())
        .build();
    }

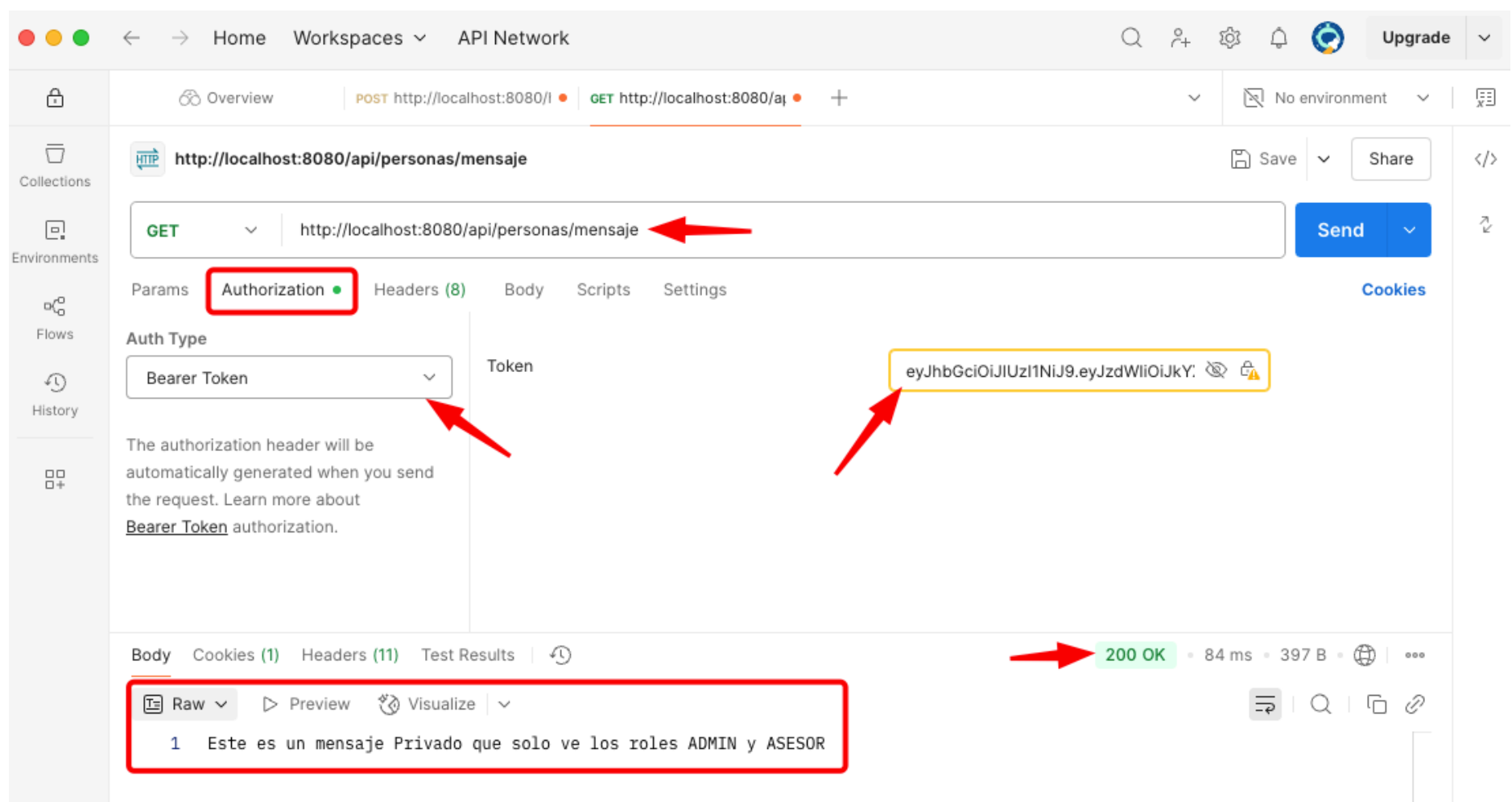
```

Ya al probar los servicios verificaremos que se genere el token y se permite la seguridad de las apis usandolo.

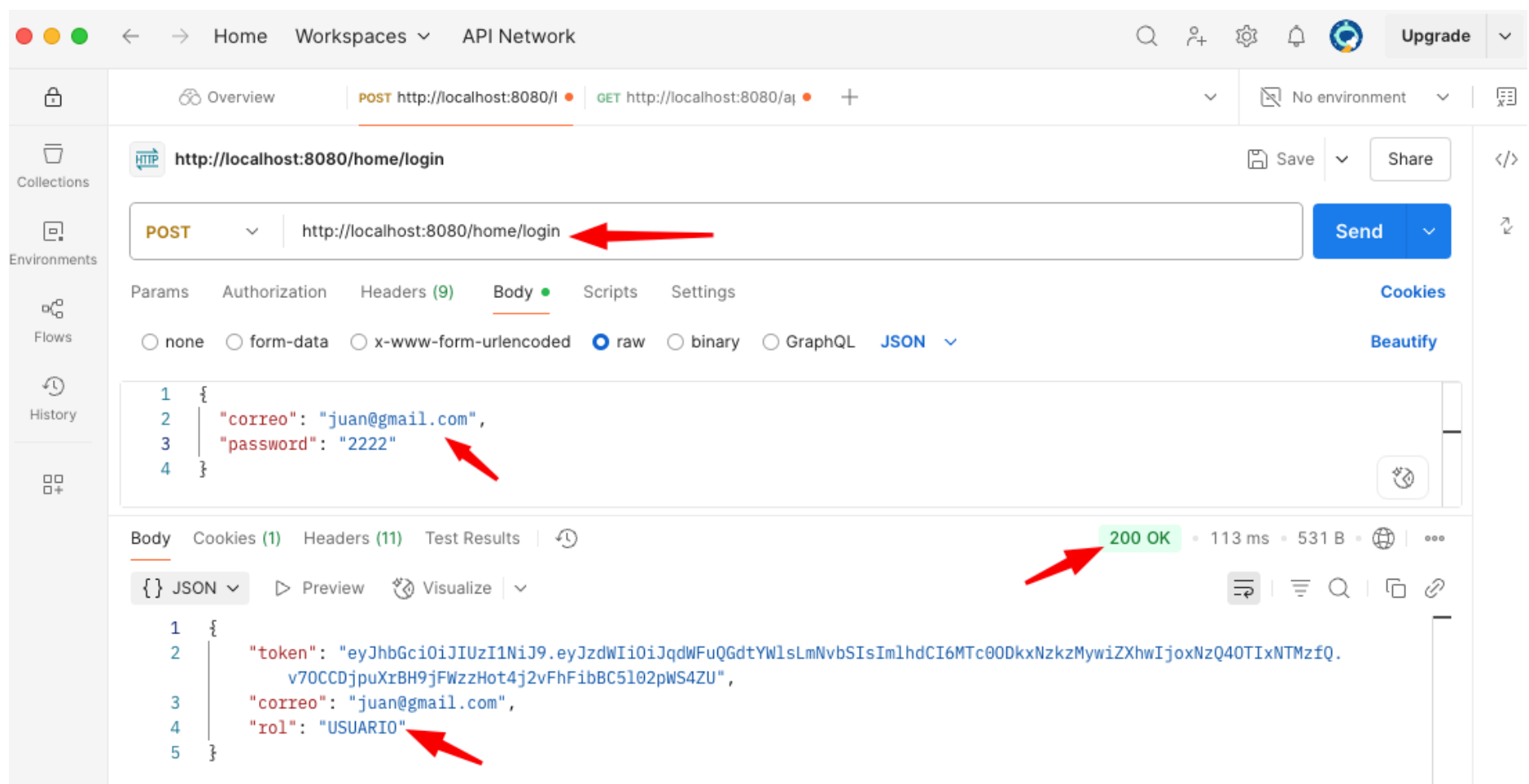
Probamos con un usuario de tipo ADMIN, primero se realiza el login y copiamos el token generado



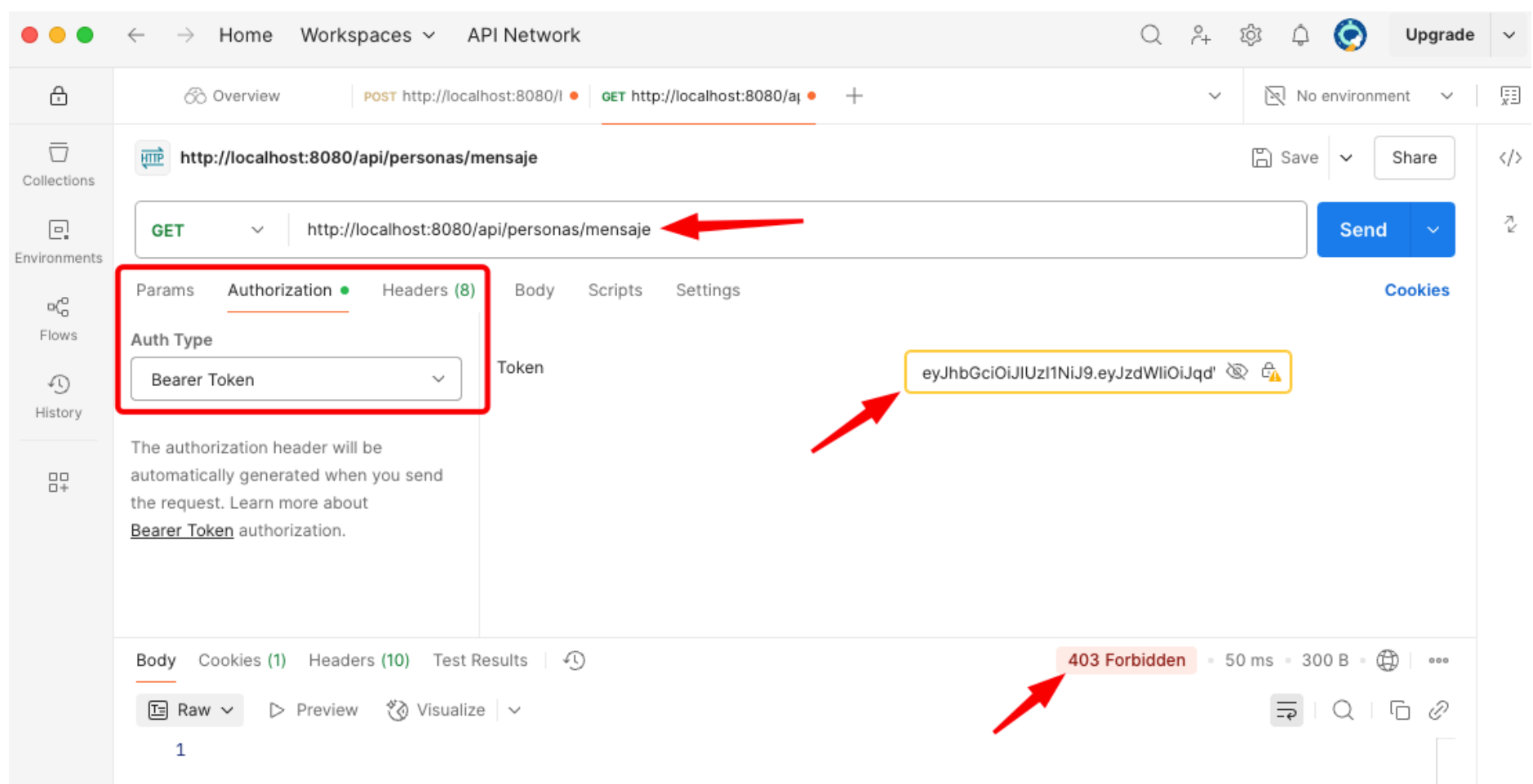
posteriormente se realiza la consulta donde validamos por medio del Token y al ser ADMIN se tiene permisos en el servicio



Ahora probemos con un usuario de tipo USUARIO, copiamos el token generado



Al ingresar al endpoint del mensaje y pegar el token, vemos que no contamos con los permisos al no ser ADMIN ni ASESOR



Pero si con el mismo token validamos uno de los endpoint con permiso de USUARIO si podremos ver la información.

The screenshot shows the API Network interface with the following details:

- Request URL:** `http://localhost:8080/api/personas/2`
- Method:** GET
- Authorization:** Bearer Token
- Token:** `eyJhbGciOiJIUzI1NiJ9.eyJzdWiiOiJqd' (highlighted with a red arrow)`
- Response Status:** 200 OK (highlighted with a red arrow)
- Response Body (JSON):**

```
1 {
2   "id": 2,
3   "nombre": "Carlos",
4   "direccion": "Avenida Siempre Viva 123",
5   "telefono": "3213456432",
6   "correo": "carlos@gmail.com",
7   "password": "$2a$10$9lQiI9o8JCvrQvPHKcfH.OcV/iU007vP3axqpzWRCuYiAh4q8zeP0",
8   "rol": "USUARIO"
9 }
```

 (The value `"id": 2,` is highlighted with a red arrow)

por ahora nos queda pendiente el proceso para el cierre de sesión y habilitar corss para el consumo de los servicios.

Instructor: Cristian David Henao Hoyos